

Parte 1: Investigación Teórica

Actividad de Investigación y Práctica: Estructuras de Datos Avanzadas y APIs con ASP.NET Core

Nombre: Josué Danilo Colindres Vidal Carnet: 202343544

1. Estructuras de Datos Eficientes

Árboles Binarios de Búsqueda (ABB)

Un **Árbol Binario de Búsqueda**, también conocido como **ABB**, es una estructura de datos jerárquica en la que cada nodo puede tener como máximo dos hijos: un hijo izquierdo y un hijo derecho.

La regla principal de ordenamiento indica que los valores menores que el nodo actual se colocan en el subárbol izquierdo, mientras que los valores mayores se colocan en el subárbol derecho.

Por ejemplo, si la raíz del árbol tiene el valor 10, entonces un valor como 5 iría al lado izquierdo, mientras que un valor como 15 iría al lado derecho. Esta organización permite buscar datos comparando el valor buscado con el nodo actual y decidiendo si se debe continuar por la izquierda o por la derecha.

Cuando el árbol está relativamente balanceado, las operaciones de búsqueda, inserción y eliminación pueden ser eficientes, alcanzando una complejidad aproximada de **$O(\log n)$** . Sin embargo, su principal desventaja aparece cuando los datos se insertan en orden secuencial, por ejemplo: 1, 2, 3, 4 y 5.

En ese caso, el árbol pierde su forma equilibrada y se convierte prácticamente en una lista enlazada. A este problema se le conoce como **degeneración del árbol**. Cuando ocurre, las operaciones pueden llegar a una complejidad de **$O(n)$** , porque en el peor caso se debe recorrer nodo por nodo.

Árboles AVL

Un **Árbol AVL** es un tipo especial de Árbol Binario de Búsqueda que se mantiene **auto-balanceado**. Esto significa que, después de insertar o eliminar un nodo, el árbol verifica si quedó desbalanceado y realiza rotaciones para recuperar el equilibrio.

El equilibrio de cada nodo se mide con el **factor de balanceo**, que compara la altura del subárbol izquierdo con la altura del subárbol derecho.

$$\text{Factor} = \text{Altura Izquierda} - \text{Altura Derecha}$$

En un árbol AVL, el factor de balanceo de cada nodo debe mantenerse entre -1, 0 y 1. Estos valores indican lo siguiente:

- **0**: ambos subárboles tienen la misma altura.
- **1**: el subárbol izquierdo es un nivel más alto.
- **-1**: el subárbol derecho es un nivel más alto.

Si el factor de balanceo se sale de ese rango, el árbol realiza rotaciones. Estas pueden ser una rotación simple a la izquierda, una rotación simple a la derecha, una rotación doble izquierda-derecha o una rotación doble derecha-izquierda.

Gracias a este auto-balanceo, la altura del árbol se mantiene controlada. Por eso, las operaciones de búsqueda, inserción y eliminación conservan una complejidad de **$O(\log n)$** . A diferencia de un ABB común,

un AVL evita degenerarse en una lista enlazada cuando los datos se insertan en un orden desfavorable.

2. Fundamentos de Web APIs

¿Qué es una API?

Una **API**, o Interfaz de Programación de Aplicaciones, es un medio que permite que dos sistemas o aplicaciones se comuniquen entre sí. En el desarrollo web, una API permite que un cliente solicite información o envíe datos a un servidor.

Por ejemplo, una API puede permitir consultar una lista de nodos, insertar un nuevo nodo, actualizar información o eliminar registros. Normalmente, las APIs web intercambian datos en formato **JSON**, porque es un formato ligero y fácil de interpretar tanto para clientes como para servidores.

Modelo Cliente-Servidor

El modelo **Cliente-Servidor** es una forma de comunicación en la que participan dos partes principales. El **cliente** es quien realiza una solicitud, como un navegador, una aplicación móvil, Postman o Bruno. El **servidor** es quien recibe la solicitud, procesa la información y devuelve una respuesta.

El proceso general funciona de la siguiente manera:

- El cliente envía una petición o **Request** al servidor.
- La petición viaja utilizando el protocolo **HTTP**.
- El servidor recibe la petición y ejecuta la lógica correspondiente.
- El servidor devuelve una **Response** con un código de estado y, si corresponde, datos en formato JSON.
- El cliente recibe la respuesta y la muestra o la procesa.

Por ejemplo, si el cliente envía una petición GET a la ruta `/api/nodos`, el servidor puede responder con una lista de nodos en formato JSON y con el código de estado 200 OK.

Verbos HTTP

GET

El método **GET** se utiliza para recuperar información desde el servidor. Su propósito principal es consultar recursos, no modificarlos. Por ejemplo, una petición GET a `/api/nodos` puede devolver todos los nodos almacenados en la API.

Este verbo es **idempotente**, lo que significa que realizar la misma petición varias veces produce el mismo efecto en el servidor. Consultar una lista de nodos una vez o diez veces no cambia la información almacenada.

GET /api/nodos

POST

El método **POST** se utiliza para crear nuevos recursos en el servidor. A diferencia de GET, este método sí envía información en el cuerpo de la petición, normalmente en formato JSON.

Por ejemplo, el cliente puede enviar un objeto con un identificador y un valor para agregar un nuevo nodo a la colección. Si la creación se realiza correctamente, el servidor puede responder con el código **201**.

Created.

POST /api/nodos

El método POST normalmente **no es idempotente**, porque si se envía la misma petición varias veces, se podrían crear varios registros repetidos o intentar insertar el mismo recurso más de una vez. Por eso se utiliza principalmente para operaciones de creación o inserción de datos.

Fuente base: Actividad 11 de Junio IPC2P - Parte 1: Investigación Teórica.